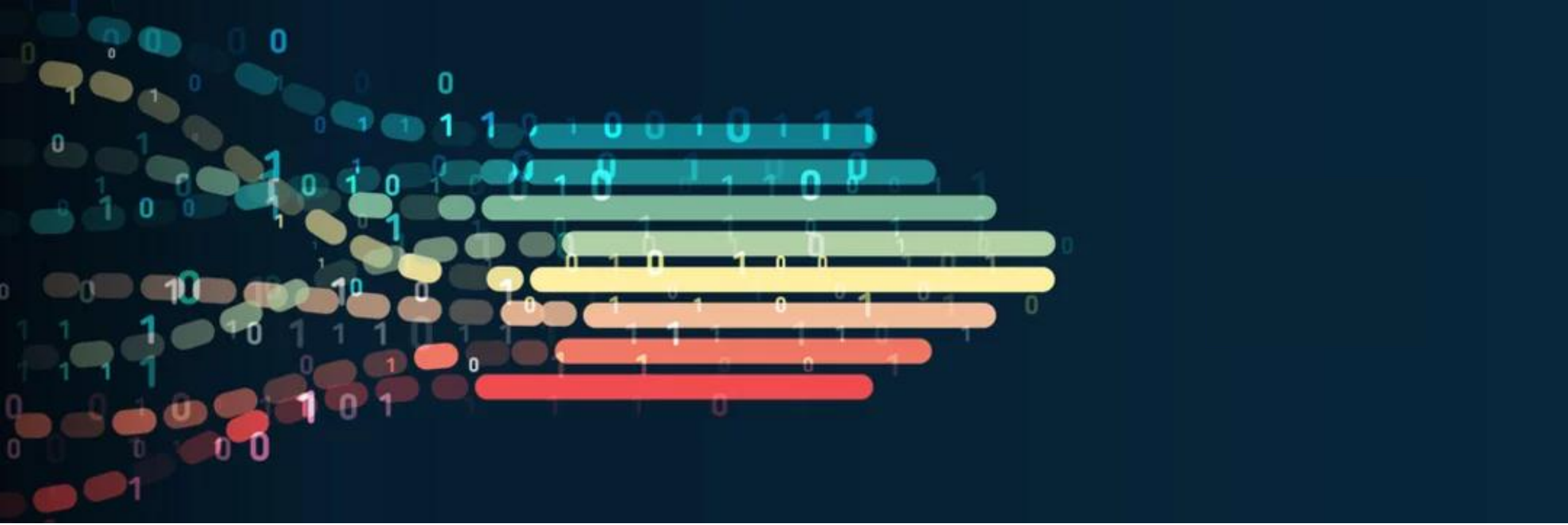


Análise de Algoritmos

Pesquisa em Largura

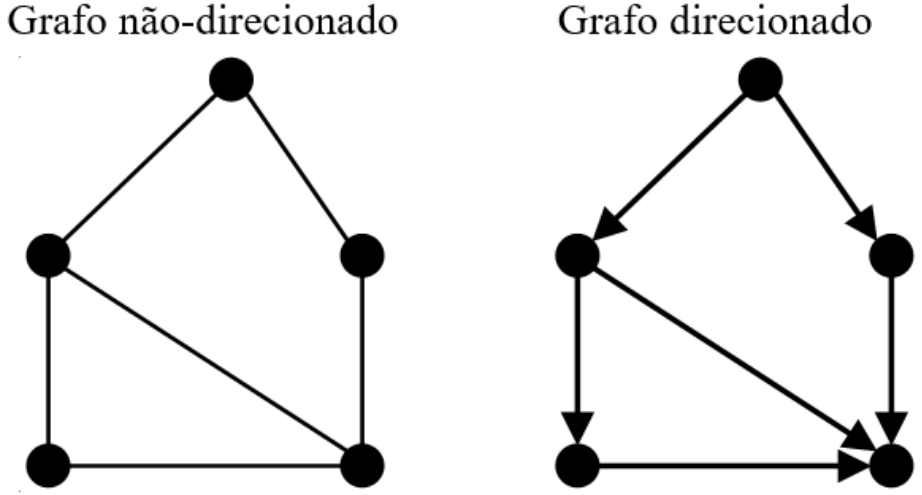
Prof. Dr. Humberto A. P. Zanetti

Roteiro

- O que são grafos?
- Algoritmo de pesquisa em largura
  - BFS (*Breadth-First Search*)
- Exemplo prático

O que são grafos?

- **Definição**: Um grafo modela conjuntos de conexões usando nodos (ou vértices) e arestas.
- **Exemplos**: mapas de trajetos, redes sociais, recomendações (“quem conhece quem”), dívidas em jogos (“quem deve para quem”).
- **Tipos**: grafos direcionados (arestas com direção) e não-direcionados.



Problemas clássicos com grafos

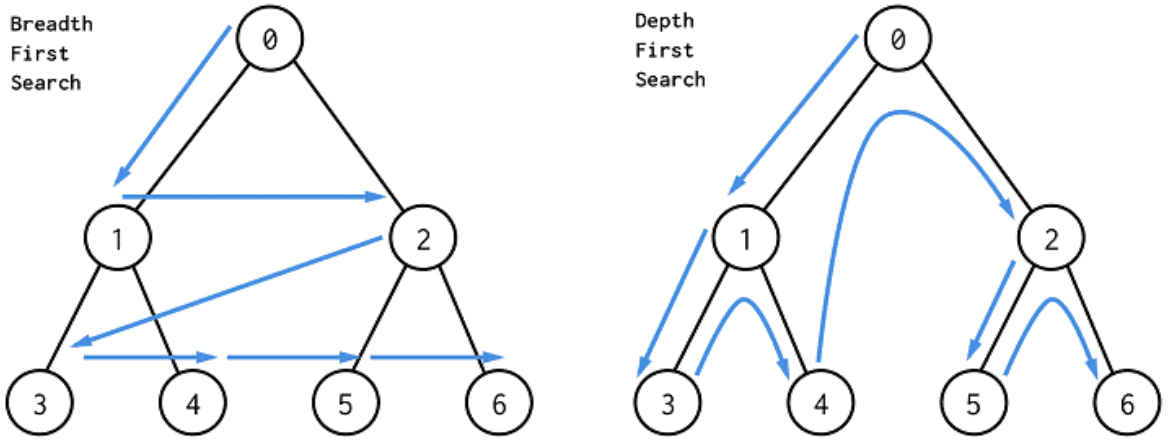
- Encontrar o caminho com o menor número de etapas (menor caminho; ex.: qual o menor número de conexões de ônibus entre dois pontos).
- Proximidade em redes (ex.: amigo mais próximo que tem determinada característica).
- Modelar jogos (ex.: menor número de jogadas até vencer).

Na Ciência de Dados...

- **Análise de Redes Sociais**: BFS é usado para encontrar conexões entre pessoas, calcular graus de separação, identificar comunidades e sugerir amigos ou conexões em plataformas sociais.
- **Recomendações e propagação de Informação**: Em sistemas de recomendação, BFS pode ser utilizado para explorar redes de usuários e itens, identificando caminhos curtos entre interesses ou propagando informações em redes de influência.
- **Exploração de Dados em grafos**: BFS é fundamental para explorar grandes bases de dados representadas como grafos, como bancos de dados de conhecimento, redes de citações acadêmicas ou relações entre entidades em textos.
- **Processamento de Linguagem Natural**: Em tarefas como análise de dependências sintáticas, BFS pode ser usado para percorrer árvores de análise e extrair relações entre palavras ou frases.
- **Classificação topológica e dependências**: BFS é utilizado para ordenar tarefas ou processos com dependências, como em pipelines de dados, workflows de ETL e análise de precedência em projetos

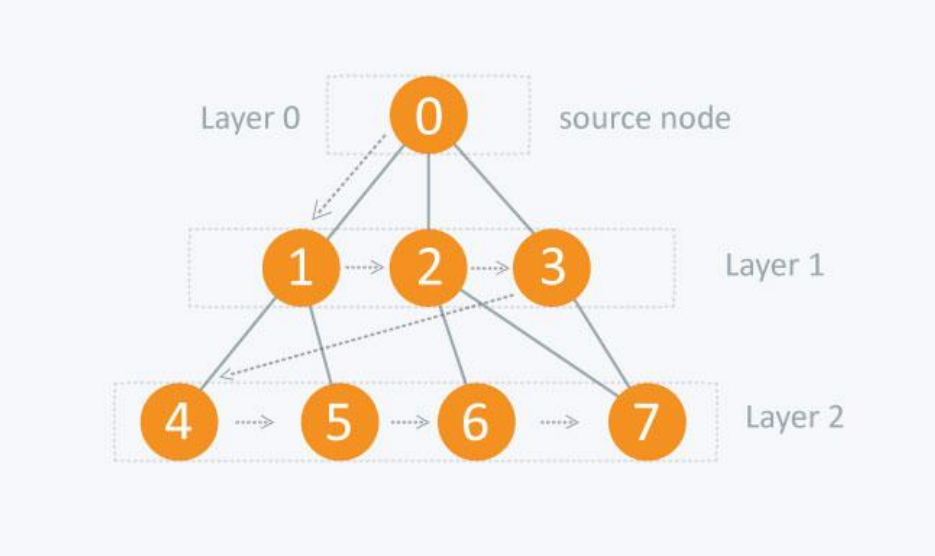
Algoritmo BFS (*Breadth-First Search*)

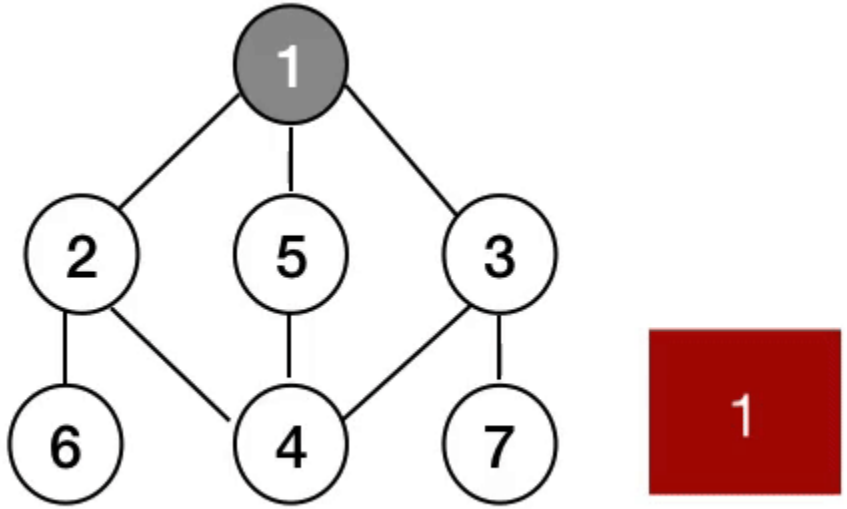
- **Objetivo**: identificar se existe conexão entre dois nós (“há caminho de A até B?”) e qual o menor caminho.
- **Ideia central**: visitar primeiro todos os vizinhos imediatos, depois os vizinhos dos vizinhos, seguindo em “largura”.
- **Estrutura de dados chave**: Fila (*queue*), pois garante ordem de visita (FIFO: *First In, First Out*).



Baseado em filas

- O algoritmo é baseado em “enfileirar” (*queue*) e “desenfilerar” (*dequeue*)





Conhecendo a estrutura “deque”

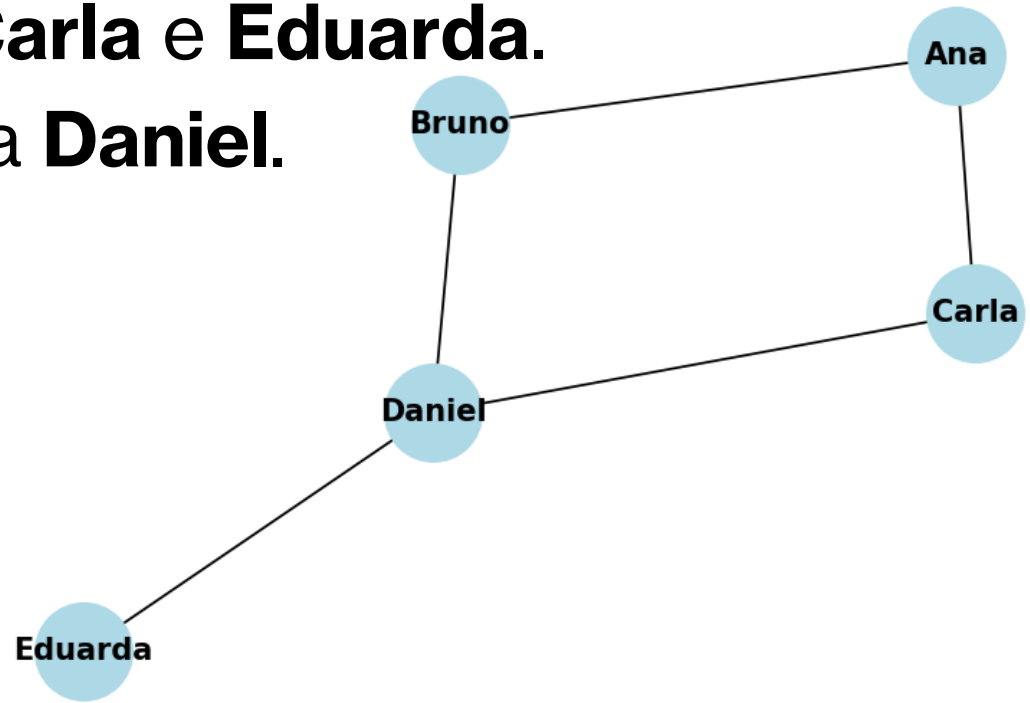
- O que é “deque”?
  - Deque (*Double-Ended Queue*) é uma estrutura de dados que permite inserção e remoção eficiente de elementos em ambas as extremidades (início e fim).
- Mais eficiente que listas para operações de fila e pilha.
- Acesso às duas pontas: Ao contrário das listas comuns, o deque é otimizado para inserções e remoções rápidas nas duas extremidades.
- Implementação de fila e pilha: Funciona como uma fila (FIFO - *First-In, First-Out*) ou uma pilha (LIFO - *Last-In, First-Out*), mas com mais flexibilidade, pois permite operações em ambos os lados.

Exemplo: caminho mais curto

- Imagine um grafo onde cada nó representa uma pessoa e cada aresta uma colaboração entre duas pessoas (ex: autores de um artigo científico juntos ou membros de um mesmo projeto).
- Aplicações
  - **Caminho mais curto entre especialistas**: Descobrir o menor número de colaborações entre dois pesquisadores, facilitando análise de redes acadêmicas.
  - **Resolvendo problemas logísticos**: Encontrar a rota mais curta entre dois pontos de distribuição em logística empresarial.
  - **Sugestões de networking**: Achar “amigos de amigos” para estratégias de recomendações automatizadas

Exemplo: caminho mais curto

- **Ana** está conectada a **Bruno** e **Carla**.
- **Bruno** está conectado a **Ana** e **Daniel**.
- **Carla** está conectada a **Ana** e **Daniel**.
- **Daniel** está conectado a **Bruno**, **Carla** e **Eduarda**.
- **Eduarda** está conectada apenas a **Daniel**.



Complexidade

- **Fila**: Usada para armazenar a ordem dos elementos a serem visitados. Para as operações temo a complexidade de *O(1)* e em busca sendo *O(n).*
  - A estrutura de dados dequeoferece complexidade de tempo *O(1)* (tempo constante) para inserção e remoção de elementos em ambas as extremidades – ou seja, tanto para **append/pop** no final quanto para **appendleft/popleft** no início da fila
- **Marcar visitados**: Necessário para evitar ciclos e verificações duplicadas.
- **Complexidade**: BFS em grafos tem tempo $O(V+E)$, onde V é o número de vértices e E de arestas.

Dúvidas!?